

Mehdi delnavazi

404198476

Code Report DAF Model

GitHub Link for Code : [GitHub Repo](#)

The dataset and PTH files for Trained models is inside google Drive

Which the link is : [Weights & Dataset](#)

Resnet Base Model

تویه این قسمت میریم سراغ توضیح کد اول که ما دیتای 10 Cifar رو رویه Resnet 18 ترین کردیم به عنوان حالت پایه . بسیاری از بخش های کد در مدل های مختلفی که ترین کردیم یکسان است و یک بار برای حالت پایه توضیح میدهم .

```
# -----  
# CONFIG  
# -----  
EPOCHS      = 20  
BATCH_SIZE  = 34  
LR           = 0.1  
PATIENCE    = 120  
DEVICE      = "cuda" if torch.cuda.is_available() else "cpu"  
SAVE_PATH   = "resnet18_daf_best.pth"  
CSV_PATH    = "resnet18_daf_history.csv"  
CHART_PATH  = "resnet18_daf_chart.png"
```

تویه این بخش ما اومدیم متغیر های مدل خودمون رو تعریف کردیم . تعداد epoch ها که با توجه به محدودیت سخت افزار ما روی 20 گذاشتیم که این 20 تا حدود 2 ساعت طول کشید.

Batch size هم 34 گذاشتیم یعنی در هر epoch 34 تا عکس ترین میشه و وزن ها بعد از دیدن 34 تا عکس اپدیت میشه. نه برای هر عکس.

و اینکه ما از GPU استفاده کردیم و برای اینکار cuda رو نصب کردیم تا بتونیم محاسبات مون رو با GPU انجام بدیم .

Resnet Base Model

```
# -----  
# 2. DATA  
# -----  
CIFAR10_MEAN = (0.4914, 0.4822, 0.4465)  
CIFAR10_STD = (0.2023, 0.1994, 0.2010)  
  
train_transform = transforms.Compose([  
    transforms.RandomCrop(32, padding=4),  
    transforms.RandomHorizontalFlip(),  
    transforms.ToTensor(),  
    transforms.Normalize(CIFAR10_MEAN, CIFAR10_STD),  
)  
  
test_transform = transforms.Compose([  
    transforms.ToTensor(),  
    transforms.Normalize(CIFAR10_MEAN, CIFAR10_STD),  
)
```

Loading...

با توجه به ابعاد کوچک تصاویر دیتابیس CIFAR-10 (32 * 32 پیکسل)، احتمال وقوع بیش‌برازش (Overfitting) در شبکه‌های عمیق بسیار بالاست.

تصویر را بزرگ‌تر کرده و برش‌های تصادفی می‌زند تا شبکه نسبت به جابه‌جایی‌های جزئی شیء (Translation Invariance) مقاوم شود.

Random Horizontal Flip(): با برگرداندن تصادفی افقی تصاویر، تنوع داده‌ها را بدون تغییر در ماهیت کلاس دوبرابر می‌کند.

نرمال‌سازی (Normalize): با کسر میانگین (**CIFAR10_MEAN**) و تقسیم بر انحراف معیار استاندارد (**CIFAR10_STD**)، توزیع آماری پیکسل‌ها را به یک توزیع نرمال با میانگین صفر و واریانس یک میل می‌دهد که این امر مانع از انفجار یا محو شدگی گرادیان (Gradient Explosion/Vanishing) در طول لایه‌ها می‌گردد.

Resnet Base Model

```
# -----
# 3. MODEL - ResNet18 adapted for 32x32 CIFAR-10
# -----
def build_resnet18_cifar():
    """
    Standard ResNet18 is designed for 224x224 (ImageNet).
    Two changes for CIFAR-10 (32x32):
    1. conv1: kernel 7→3, stride 1 (reduces spatial size)
    2. maxpool → Identity (removes early downsampling)
    """
    model = models.resnet18(weights=None)
    model.conv1 = nn.Conv2d(3, 64, kernel_size=3,
                             stride=1, padding=1, bias=False)
    model.maxpool = nn.Identity()
    model.fc = nn.Linear(512, 10)
    return model
```

شبکه استاندارد ResNet 18 در اصل برای تصاویر بزرگ دیتابیس ImageNet (224 * 224 پیکسل) طراحی شده است. اگر همان شبکه بدون تغییر روی تصاویر 32*32 اعمال شود، ابعاد فضایی نقشه ویژگی در همان لایه‌های ابتدایی به شدت کوچک شده و اطلاعات ساختاری تصویر نابود می‌شود. در تابع `build_resnet18_cifar` دو اصلاح صورت گرفته است:

1. اصلاح لایه اول کانولوشنی (`model.conv1`): اندازه فیلتر لایه اول از 7*7 با گام (Stride) ۲ به اندازه 3*3 با گام ۱ کاهش یافته است. این تغییر تضمین می‌کند که نقشه ویژگی اولیه بدون کاهش مفرط ابعاد، وارد لایه‌های بعدی شود.
2. غیرفعال‌سازی (`model.maxpool`): این لایه با بلوک همانی (`nn.Identity()`) جایگزین شده است. با این کار، لایه کاهش ابعاد فضایی ثانویه در ابتدای شبکه حذف شده و وضوح تصویر برای استخراج بافت‌های ریز حفظ می‌گردد.
3. لایه کاملاً متصل نهایی (`model.fc`): خروجی لایه خطی نهایی از ۱۰۰۰ کلاس (ImageNet) متناسب با ماهیت دیتابیس جاری به ۱۰ کلاس نگاشت یافته است.

Resnet model Base Model

```
def train_one_epoch(model, loader, criterion, optimizer):
    model.train()
    total_loss, correct, total = 0.0, 0, 0
    for inputs, labels in loader:
        inputs, labels = inputs.to(DEVICE), labels.to(DEVICE)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * inputs.size(0)
        correct += outputs.max(1)[1].eq(labels).sum().item()
        total += inputs.size(0)
    return total_loss / total, 100.0 * correct / total
```

در این بخش ما یک تابع برای یک ایپاک نوشتیم که میایم دیتا هارو برای هر batch میدیم بهش این بخش ما اول `model.train()` رو صدا میزنیم که بفهمیم که تویه حالت train هستیم که dropout ها , و batch normalisation اعمال بشه ما فقط تو حالت train اینارو فعال میکنیم و در حالت تست دیگه اینو فعال نمیشن .

Input هارو به مدل میدیم که مدل برای این batch شروع به ترین کردن بکنه و تابع خطا تعریف شده .

Resnet model Base Model

```
def evaluate(model, loader, criterion):
    model.eval()
    total_loss, correct, total = 0.0, 0, 0
    with torch.no_grad():
        for inputs, labels in loader:
            inputs, labels = inputs.to(DEVICE), labels.to(DEVICE)
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            total_loss += loss.item() * inputs.size(0)
            correct += outputs.max(1)[1].eq(labels).sum().item()
            total += inputs.size(0)
    return total_loss / total, 100.0 * correct / total
```

دیتالودر تست (`test_loader`) تصاویر را ۳۴ تا ۳۴ تا به مدل می‌دهد، مدل بدون ذخیره اطلاعات اضافی در حافظه، کلاس آن‌ها را پیش‌بینی می‌کند و در نهایت، میانگین کل خطای تست و درصد دقت نهایی (`Test Accuracy`) را به عنوان خروجی بازمی‌گرداند.

Resnet model Base Model

در مجموع مدل روی دیتا ها ترین شده و نمودار های `train & loss` رو هم به کمک تابع `Save Chart` () این تابع نمودارها رو رسم میکنه و رد نهایت یک فایل خروجی عکس داده میشه برای نمودار ها برای هر epoch هم دیتا های `accuracy` هم تویه فایل `csv` ذخیره میشن . و وزن مدل ترین شده هم خروجی داده میشه . `pth file`

و من فایل ورن های شبکه رو تویه `google drive` قرار دادم . به همراه فایل کد ها در فولدر کد یک فایل هم برای `requirement.txt` هست که بتونین `dependency` ها رو نصب کنید . یک فایل `gradcam` هم برای هر مدل ساخته شده که میاد و `gradcam` رو میکشه برای چند نمونه تا بتونیم ببینیم که توجه مدل بیشتر روی کدام بخش بوده ایا درست توجه کرده ؟ تویه بخش بعدی درباره `grad cam` و کدش یک توضیحی میدیم و برای همه مدل ها به همین ترتیب ساخته شده .

برای محاسبه ی `GradCam` ما سه تا مرحله باید طی کنیم . `Grad cam` درواقع میاد نشون میده کدوم فیچر مپ ها تاثیر بیشتری تویه `loss` دارن و هرچقدر تاثیرش بیشتر باشه اون فیچر مپ یعنی اهمیت بالایی داره .

برای محاسبه این نقشه، الگوریتم ۳ گام ریاضی طی می کند:

استخراج ویژگی ها (Activations): وقتی تصویر وارد شبکه می شود، خروجی آخرین لایه کانولوشنی (در کد شما: `model.layer4[1].conv2`) را ذخیره می کنیم.

محاسبه گرادیان و اهمیت هر کانال (Weights): شبکه کلاس نهایی (مثلاً گربه) را پیش بینی می کند و یک امتیاز (y^c) به آن می دهد. ما از این امتیاز نسبت به تمام پیکسل های ماتریس ویژگی لایه کانولوشنی مشتق (گرادیان) می گیریم سپس برای اینکه بفهمیم کل یک کانال چقدر اهمیت دارد، روی بعد فضایی میانگین گیری (`Global Average Pooling`) می کنیم تا وزن هر کانال به دست آید.

```
# GRAD-CAM
#
class GradCAM:
    def __init__(self, model, target_layer):
        self.model = model
        self.activation = None
        self.gradient = None
        self._fwd = target_layer.register_forward_hook(self._save_act)
        self._bwd = target_layer.register_full_backward_hook(self._save_grad)

    def _save_act(self, module, input, output):
        self.activation = output.detach()

    def _save_grad(self, module, grad_input, grad_output):
        self.gradient = grad_output[0].detach()

    def generate(self, inp):
        self.model.eval()
        output = self.model(inp)
        pred = output.argmax(dim=1).item()
        conf = output.softmax(dim=1)[0, pred].item()

        self.model.zero_grad()
        output[0, pred].backward()

        weights = self.gradient.mean(dim=(2, 3), keepdim=True)
        cam = torch.relu((weights * self.activation).sum(dim=1)).squeeze()
        cam = cam.cpu().numpy()
        cam -= cam.min()
        if cam.max() > 0:
            cam /= cam.max()

        return cam, pred, conf
```

تحلیل عملکرد ساختاری:

- **register_forward_hook:** این متد، تابع `_save_act` را به لایه هدف (`target_layer`) متصل می‌کند. در طول فاز ارزیابی، به محض اینکه تانسور از لایه هدف عبور کند، این تابع فراخوانی شده و کپی کالبره‌شده‌ای از نقشه ویژگی‌ها (Feature Maps) را بدون درگیر کردن گراف محاسباتی (`.detach()`) در متغیر `self.activation` ذخیره می‌کند.
- **register_full_backward_hook:** این متد، تابع `_save_grad` را در گراف پس‌انتشار ثبت می‌کند. هنگامی که مشتقات خطا به سمت عقب حرکت می‌کنند، گرادیان‌های خطا نسبت به خروجی لایه هدف صید شده و در متغیر `self.gradient` ذخیره می‌شوند.

استخراج و فرموله‌سازی نقشه توجه (CAM Generation)

هسته اصلی پردازش تصویر در متد generate اجرا می‌شود، جایی که منطق ریاضی الگوریتم Grad-CAM ماتریکس توجه را استخراج می‌کند.

```
def generate(self, inp):
    self.model.eval()
    output = self.model(inp)
    pred = output.argmax(dim=1).item()
    conf = output.softmax(dim=1)[0, pred].item()

    # محاسبه گرادیان‌ها نسبت به کلاس پیش‌بینی‌شده
    self.model.zero_grad()
    output[0, pred].backward()
```

تبیین ریاضی و منطق خطوط:

1. استخراج کلاس بیشینه: پس از عبور تصویر ورودی (inp) از شبکه در فاز ارزیابی (self.model.eval())، اندیس پویای کلاس دارای بالاترین امتیاز (pred) و میزان قطعیت مدل (conf) استخراج می‌گردد.
2. محاسبه مشتق موضعی: با فراخوانی متد output[0, pred].backward()، مشتق امتیاز کلاس پیش‌بینی‌شده (Y^c) نسبت به نقشه‌های ویژگی لایه هدف A^k در گام معکوس محاسبه می‌شود

$$\frac{\partial Y^c}{\partial A^k}$$

این گرادیان‌ها در متغیر self.gradient که پیش‌تر توسط قلاب عقبگرد قفل شده بود، ذخیره می‌شوند.

```
# محاسبه وزن کاتال‌ها با میانگین‌گیری فضایی
weights = self.gradient.mean(dim=(2, 3), keepdim=True)

# ترکیب خطی وزن‌ها و نقشه‌های فعال‌سازی به همراه اعمال تابع ReLU
cam = torch.relu((weights * self.activation).sum(dim=1)).squeeze()

# نرمال‌سازی ماتریکس در بازه [0, 1]
cam = cam.cpu().numpy()
cam -= cam.min()
if cam.max() > 0:
    cam /= cam.max()

return cam, pred, conf
```

محاسبه اهمیت کانال‌ها: (Global Average Pooling) گرادیان‌های صید شده دارای ابعاد $[B, C, H, W]$ هستند. با اعمال عملگر میانگین‌گیری بر روی ابعاد فضایی H و W ، یک بردار وزن به دست می‌آید که نشان‌دهنده میزان اهمیت و وزن هر فیلتر برای کلاس هدف است. این فرآیند از فرمول ریاضی زیر تبعیت می‌کند:

$$\alpha_k^c = \frac{1}{Z} \sum_i \sum_j \frac{\partial Y^c}{\partial A_{i,j}^k}$$

در پایتورچ، دستور `self.gradient.mean(dim=(2, 3))` دقیقاً معادل عملگر ریاضی فوق $\frac{1}{Z} \sum_i \sum_j$ است.

ترکیب خطی و فیلترینگ صعودی: (Linear Combination & ReLU) وزن‌های حاصل در ماتریکس ویژگی‌های ذخیره‌شده ضرب شده و روی بعد کانال ($\text{dim}=1$) با یکدیگر جمع می‌شوند. در نهایت، تانسور از یک لایه `torch.relu` عبور داده می‌شود. اعمال تابع ReLU نقشی کلیدی دارد؛ زیرا این تابع ویژگی‌هایی را که تأثیر منفی یا کاهنده روی امتیاز کلاس هدف دارند (پیکسل‌هایی که امتیاز آن‌ها کمتر از صفر است) حذف کرده و تنها ویژگی‌های مثبت و تقویت‌کننده کلاس را حفظ می‌کند

$$L_{\text{Grad-CAM}}^c = \text{ReLU} \left(\sum_k \alpha_k^c A^k \right)$$

موازی‌سازی و نرمال‌سازی ماتریکس: نقشه حاصل پس از انتقال به حافظه میزبان و تبدیل به آرایه نام‌پای، با تفریق مقدار کمینه و تقسیم بر مقدار بیشینه، کاملاً در بازه استاندارد $[0, 1]$ هنجار می‌شود تا به عنوان یک نقشه حرارتی استاندارد، آماده انطباق و ترسیم روی تصویر اصلی گردد.

SE Model

نوآوری بلوک SE این است که به شبکه یاد می‌دهد به صورت پویا و وابسته به خود تصویر ورودی، به کانال‌های مهم‌تر وزن بیشتر (نزدیک به ۱) و به کانال‌های کم‌اهمیت یا نویزی وزن کمتر (نزدیک به ۰) اختصاص دهد.

بلوک SE یک ماژول توجه کانالی (Channel Attention) است. در شبکه‌های معمولی، تمام کانال‌های خروجی یک لایه ارزش یکسانی دارند؛ اما بلوک SE با محاسبه روابط بین کانال‌ها، به صورت پویا به کانال‌های حیاتی تصویر وزن بزرگ‌تر (نزدیک به ۱) و به کانال‌های نویزی وزن کوچک‌تر (نزدیک به ۰) اختصاص می‌دهد. پیاده‌سازی این بلاک در کدهای پایتورچ شامل دو بخش اصلی است: تعریف لایه‌ها (`__init__`) و جریان محاسباتی داده‌ها (`forward`).

بخش اول: معماری و تعریف لایه‌ها (`__init__`)
در ابتدا لایه‌های مورد نیاز برای ۳ گام اصلی SE (Squeeze, Excite, Scale) تعریف می‌شوند:

```
# -----
# 3. SE BLOCK
# -----
class SEBlock(nn.Module):
    """
    Squeeze-and-Excitation block.
    reduction=16 is the standard from the original SE-Net paper.

    Steps:
    1. Squeeze   : Global Avg Pool → one number per channel
    2. Excite    : Two FC layers learn channel importance weights
    3. Scale     : Multiply weights back into the feature map
    """
    def __init__(self, channels, reduction=16):
        super().__init__()
        self.squeeze = nn.AdaptiveAvgPool2d(1)          # [C,H,W] → [C,1,1]
        self.excitation = nn.Sequential(
            nn.Flatten(),                               # [C,1,1] → [C]
            nn.Linear(channels, channels // reduction, bias=False),
            nn.ReLU(inplace=True),
            nn.Linear(channels // reduction, channels, bias=False),
            nn.Sigmoid()                                # weights 0-1
        )

    def forward(self, x):
        b, c, _, _ = x.shape
        weights = self.squeeze(x)                       # [B, C, 1, 1]
        weights = self.excitation(weights)              # [B, C]
        weights = weights.view(b, c, 1, 1)             # [B, C, 1, 1]
        return x * weights                             # scale channels
```

تحلیل لایه‌های تعریف شده:

۱. `self.squeeze = nn.AdaptiveAvgPool2d(1):`

این لایه وظیفه فشردن فضای اطلاعات فضایی تصویر $H \times W$ را بر عهده دارد. با استفاده از فرآیند Global Average Pooling، میانگین تمام پیکسل‌های هر کانال را می‌گیرد تا از هر کانال فقط یک عدد به عنوان خلاصه آماری باقی بماند. عدد خروجی مستقل از اندازه تصویر ورودی، همیشه ابعاد فضایی 1×1 خواهد داشت.

۲. بلاک: `self.excitation = nn.Sequential(...):`

این بخش حاوی یک شبکه عصبی کوچک تمام متصل (Shared MLP) با ساختار گلوگاه (Bottleneck) است تا روابط متقابل کانال‌ها را یاد بگیرد:

- **nn.Flatten():** تانسور ۴ بعدی حاصل از اسکوئیز را به یک ماتریس ۲ بعدی تبدیل می‌کند تا لایه‌های خطی بتوانند آن را پردازش کنند.
- **لایه خطی اول (nn.Linear):** تعداد کانال‌ها را به ضریب کاهش (reduction=16) فشرده می‌کند (مثلاً ۵۱۲ کانال تبدیل به ۳۲ نورون می‌شود). این کار از اورفیت شدن جلوگیری کرده و بار محاسباتی (Overhead) را به شدت کاهش می‌دهد.
- **nn.ReLU(inplace=True):** روابط غیرخطی و پیچیده را به محاسبات اهمیت کانال‌ها اضافه می‌کند.
- **لایه خطی دوم (nn.Linear):** ابعاد فشرده شده را مجدداً به تعداد کانال‌های اصلی بازمی‌گرداند تا برای هر کانال یک وزن اختصاصی داشته باشیم.
- **nn.Sigmoid():** حیاتی‌ترین لایه این بخش است. خروجی را بین بازه ۰ و ۱ مهار می‌کند. وزن متمایل به ۱ یعنی کانال بسیار مهم است و وزن متمایل به ۰ یعنی کانال باید حذف یا تضعیف شود.

بخش دوم: جریان محاسباتی داده‌ها (forward)

در این بخش، تانسور ورودی x با ابعاد $[B, C, H, W]$ خط به خط وارد عملیات ریاضی می‌شود:

```
def forward(self, x):
    b, c, _, _ = x.shape
    weights = self.squeeze(x)           # [B, C, 1, 1]
    weights = self.excitation(weights)  # [B, C]
    weights = weights.view(b, c, 1, 1)  # [B, C, 1, 1]
    return x * weights                 # scale channels
```

تحلیل گام به گام کدهای محاسباتی:

گام اول: اجرای Squeeze

`weights = self.squeeze(x)`

برای هر کانال c ، میانگین کل پیکسل‌های فضایی محاسبه می‌شود:

$$z_c = \frac{1}{H \times W} \sum_{i=1}^H \sum_{j=1}^W x_c(i, j)$$

تانسور ورودی $[B, C, H, W]$ به ابعاد $[B, C, 1, 1]$ می‌رسد.

گام دوم: اجرای Excitation

`weights = self.excitation(weights)`

بردار خلاصه ویژگی‌ها وارد شبکه گلوگاه شده، فشرده و بازسازی می‌شود و در نهایت پس از عبور از سگموئید، بردار وزن‌ها را به ابعاد $[B, C]$ تولید می‌کند. فرمول ریاضی این گام به این صورت است:

$$s = \sigma(W_2 \cdot \text{ReLU}(W_1 \cdot z))$$

که در آن W_1 وزن لایه کاهنده و W_2 وزن لایه افزایشنده است.

گام سوم: تراز کردن ابعاد و اعمال مقیاس دهی (Scale)

View : چون تانسور اصلی تصویر x چهاربعدی است $([B, C, H, W])$ و ماتریس وزن ها دوبعدی است $([B, C])$ ، پایتورچ نمی تواند آن ها را در هم ضرب کند. با دستور `view(b, c, 1, 1)` ابعاد وزن ها را مجدداً به صورت مجازی چهاربعدی می کنیم تا قابلیت Broadcasting فعال شود.

ضرب نهایی: $(x * weights)$ در این خط، هر کانال از تصویر ورودی به طور کامل در وزن مربوط به خودش ضرب می شود:

$$\hat{x}_c = s_c \cdot x_c$$

CBAM (Convolutional Block Attention Module)

بلاک CBAM برخلاف SE-Net که فقط روی کانال ها کار می کرد، یک مکانیسم توجه دوگانه و متوالی (Sequential) است. این بلاک به دو سوال اساسی پاسخ می دهد:

1. توجه کانالی: «**Channel Attention**» کدام ویژگی معنایی (چه چیزی) مهم است؟
2. توجه مکانی: «**Spatial Attention**» آن ویژگی معنایی در کجای تصویر (کجا) قرار دارد؟

بخش اول class Channel Attention: کدام کانال مهم است؟

```
# -----
class ChannelAttention(nn.Module):
    def __init__(self, channels, reduction=16):
        super(ChannelAttention, self).__init__()

        self.avg_pool = nn.AdaptiveAvgPool2d(1)
        self.max_pool = nn.AdaptiveMaxPool2d(1)

        # Shared MLP - same weights used for both avg and max
        self.mlp = nn.Sequential(
            nn.Flatten(),
            nn.Linear(channels, channels // reduction, bias=False),
            nn.ReLU(inplace=True),
            nn.Linear(channels // reduction, channels, bias=False)
        )

        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        avg_out = self.mlp(self.avg_pool(x))          # AvgPool -> MLP
        max_out = self.mlp(self.max_pool(x))          # MaxPool -> MLP
        scale = self.sigmoid(avg_out + max_out)        # add and sigmoid
        return x * scale.unsqueeze(2).unsqueeze(3)    # multiply with feature map
```

هدف این زیرماژول، محاسبه روابط متقابل بین کانال‌های ویژگی و استخراج نقشه توجه کانالی $M_c \in \mathbb{R}^{C \times 1 \times 1}$ از تانسور ورودی $X \in \mathbb{R}^{B \times C \times H \times W}$ است. نوآوری CBAM نسبت به معماری‌های پیشین، بهره‌گیری هم‌زمان از دو عملگر (Global Average Pooling) و (Global Max Pooling) به منظور حفظ پویایی و اطلاعات آماری غنی‌تر است.

self.avg_pool(x): و **self.max_pool(x):** تانسور ورودی x با ابعاد $[B, C, H, W]$ به طور موازی وارد این دو لایه می‌شود. خروجی این لایه‌ها تانسورهای ۴ بعدی با ابعاد $[B, C, 1, 1]$ هستند که ابعاد فضایی آن‌ها فشرده شده است.

self.mlp(...): هر دو z به صورت مجزا وارد شبکه پیش‌خور مشترک می‌شوند. لایه `nn.Flatten()` ابتدا ابعاد $[B, C, 1, 1]$ را به ماتریس دو بعدی $[B, C]$ تبدیل می‌کند. کاهش بعد محاسباتی با ضرب گلوگاه `reduction=16` در لایه خطی اول انجام شده و پس از اعمال تابع غیرخطی `ReLU`، ابعاد بردار در لایه خطی دوم مجدداً به تعداد کانال‌های اصلی (`channels`) بازمی‌گردد. پارامترهای این شبکه بین هر دو مسیر مشترک (`Shared`) هستند.

self.sigmoid(avg_out + max_out): خروجی‌های دو مسیر به صورت المان‌به‌المان جمع شده و توسط تابع سگموئید در بازه $[0, 1]$ مهار می‌شوند تا بردار وزن پویا برای هر کانال حاصل شود.

scale.unsqueeze(2).unsqueeze(3): برای امکان پذیر شدن ضرب تانسوری در ورودی اصلی ۴ بعدی، دو بعد همانی به انتهای بردار وزن اضافه شده تا ابعاد آن به $[B, C, 1, 1]$ تغییر یابد و از طریق مکانیزم `Broadcasting` در تانسور ویژگی ضرب شود.

۲. زیرماژول توجه مکانی (Spatial Attention)

این زیرماژول بر روابط هندسی پیکسل‌ها تمرکز دارد و به سؤال «آن ویژگی معنایی در کجای تصویر قرار دارد؟» پاسخ می‌دهد.

```

class SpatialAttention(nn.Module):
    def __init__(self, kernel_size=7):
        super(SpatialAttention, self).__init__()
        # ورودی ۲ کانال (میانگین و بیشینه) و خروجی ۱ کانال وزن مکانی است
        self.conv = nn.Conv2d(2, 1, kernel_size=kernel_size, padding=kernel_size // 2)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        avg_out = torch.mean(x, dim=1, keepdim=True) # ابعاد خروجی: [B, 1, H, W]
        max_out = torch.max(x, dim=1, keepdim=True)[0] # ابعاد خروجی: [B, 1, H, W]
        concat = torch.cat([avg_out, max_out], dim=1) # الحاق کانالی: [B, 2, H, W]
        scale = self.sigmoid(self.conv(concat)) # استخراج ماتریکس فضایی: [B, 1, H, W]
        return x * scale # ضرب در تانسور ورودی

```

torch.mean و **torch.max**: فشرده‌سازی این بار در طول بعد کانال ($\text{dim}=1$) انجام می‌شود. پارامتر $\text{keepdim}=\text{True}$ ساختار تانسور را به صورت ۴ بعدی حفظ می‌کند. خروجی این دو خط، دو نقشه دوبعدی با ابعاد $[B, 1, H, W]$ است که خلاصه‌ای از هیستوگرام فضایی کانال‌هاست.

torch.cat(..., dim=1): این دو نقشه فضایی روی بعد کانال به یکدیگر متصل شده و تانسوری با ابعاد $[B, 2, H, W]$ تشکیل می‌دهند.

self.conv(concat): یک لایه کانولوشن با هسته بزرگ 7×7 بر روی تانسور اعمال می‌شود. استفاده از فیلتر بزرگ 7×7 برای تضمین وسعت میدان دید (Receptive Field) و درک همسایگی پیکسل‌ها حیاتی است. پدینگ لایه به صورت هوشمند برابر با $\text{kernel_size} // 2$ تنظیم شده تا ابعاد فضایی H و W دست‌نخورده باقی بماند و تانسور خروجی به ابعاد $[B, 1, H, W]$ تقلیل یابد.

x * scale: خروجی کانولوشن پس از عبور از سگموئید، به صورت المان‌به‌المان در تانسور ورودی ضرب می‌شود تا نواحی نویزی تصویر تضعیف و کانون هندسی الگوها تقویت شود.

بلوک یکپارچه و متوالی (CBAM (The Final CBAM Block)

این کلاس به عنوان ماژول اصلی، دو زیرماژول فوق را به صورت یک زنجیره متوالی به هم متصل می‌کند. تانسور ویژگی ورودی ابتدا از فیلتر کانال عبور کرده، کالیبره می‌شود و سپس خروجی اصلاح‌شده‌ی آن مستقیماً به عنوان ورودی لایه توجه مکانی ارسال می‌گردد.

```

class CBAM(nn.Module):
    def __init__(self, channels, reduction=16, kernel_size=7):
        super(CBAM, self).__init__()
        self.channel_att = ChannelAttention(channels, reduction)
        self.spatial_att = SpatialAttention(kernel_size)

    def forward(self, x):
        x = self.channel_att(x) # گام اول: اصلاح و کالیبراسیون بعد کانال (چه چیزی؟)
        x = self.spatial_att(x) # گام دوم: اصلاح و کالیبراسیون بعد فضایی (کجا؟)
        return x

```


معماری پیشنهادی شبکه با مکانیزم ادغام پویای توجه (DAF)

معماری مرسوم CBAM توجه کانالی و مکانی را به صورت زنجیره‌ای متوالی و با ضریب وزنی ثابت (ثابت ۱.۰) به تانسور ویژگی اعمال می‌کند. این رویکرد فرض می‌کند که همواره هر دو نوع توجه به یک اندازه برای تمام لایه‌ها و تانسورهای ورودی حیاتی هستند.

در این پژوهش، به منظور بهینه‌سازی محاسباتی و هوشمندسازی فرآیند فیلترینگ الگوها، معماری ادغام پویای توجه (DAF) یا (Dynamic Attention Fusion) پیاده‌سازی شده است. نوآوری اصلی این ساختار، تعبیه یک زیرسیستم مستقل تحت عنوان امتیازدهی پویای پیچیدگی (Complexity Scorer) است که تانسور ویژگی را در لحظه تحلیل کرده و تعیین می‌کند که شبکه در این لایه مشخص، به چه میزانی از توجه کانالی و چه میزانی از توجه مکانی نیاز دارد.

۱. زیرماژول امتیازدهی پویای پیچیدگی (Complexity Scorer)

این بلاک نوآورانه وظیفه دارد با استخراج مشخصه‌های آماری تانسور ورودی $X \in \mathbb{R}^{B \times C \times H \times W}$ ، دو امتیاز وزنی پویا شامل $score_c$ (میزان نیاز به توجه کانالی) و $score_s$ (میزان نیاز به توجه مکانی) را در بازه $[0, 1]$ محاسبه کند.

منطق آماری و تبیین ریاضی:

منطق این زیرماژول بر پایه واریانس فضایی (Spatial Variance) کانال‌ها استوار است:

- **واریانس بالا در فیلترها:** به این معنی است که اکتیویشن‌ها (فعال‌سازی‌ها) در طول ابعاد هندسی تصویر تفاوت بسیار زیادی با یکدیگر دارند. این فاکتور نشان‌دهنده وجود یک شیء یا ویژگی با مرزهای مشخص در یک مختصات جغرافیایی خاص است؛ بنابراین مدل نتیجه می‌گیرد که به **توجه مکانی (Spatial Attention)** بالایی نیاز دارد.
- **واریانس پایین در فیلترها:** به این معنی است که اکتیویشن‌ها به طور یکنواخت در کل ابعاد تصویر پخش شده‌اند و شیء موقعیت هندسی متمایزی ندارد؛ در این حالت، ویژگی‌های معنایی کل تصویر اهمیت دارند و شبکه به این نتیجه می‌رسد که **توجه کانالی (Channel Attention)** کفایت می‌کند.

```

class ComplexityScorer(nn.Module):
    def __init__(self, channels):
        super().__init__()

        # MLP takes per-channel mean and variance
        # input size = channels * 2 (mean + variance for each channel)
        self.mlp = nn.Sequential(
            nn.Linear(channels * 2, channels // 2, bias=False),
            nn.ReLU(inplace=True),
            nn.Linear(channels // 2, 2, bias=False), # 2 outputs: score_c, score_s
            nn.Sigmoid() # both between 0 and 1
        )

    def forward(self, x):
        # x shape: [B, C, H, W]
        # compute per-channel mean and variance across spatial dimensions
        mean = x.mean(dim=[2, 3]) # [B, C] average per channel
        var = x.var(dim=[2, 3]) # [B, C] variance per channel

        # concat mean and variance → [B, C*2]
        stats = torch.cat([mean, var], dim=1)

        # MLP predicts two scores
        scores = self.mlp(stats) # [B, 2]

        score_c = scores[:, 0] # how much Channel Attention
        score_s = scores[:, 1] # how much Spatial Attention

        return score_c, score_s

```

در متد forward، میانگین فضایی ($x.mean(dim=[2, 3])$) و واریانس فضایی ($x.var(dim=[2, 3])$) برای تک تک کانال‌ها استخراج شده و پس از الحاق در قالب یک بردار آماری به ابعاد $[B, C*2]$ ، به یک شبکه پیش‌خور خطی (MLP) تزریق می‌شوند تا دو بردار توجه استخراج گردند.

۲. بلوک ادغام پویای توجه (DAF Block)

این بلاک به عنوان هسته مرکزی مدیریت توجه، خروجی‌های لایه‌های کلاسیک توجه کانالی و مکانی را به صورت هوشمند و وزن‌دار ترکیب (Fuse) می‌کند. برخلاف CBAM سنتی که جریان داده در آن کاملاً متوالی و اجباری است، در اینجا جریان به صورت موازی و مشروط هدایت می‌شود.

```

class DAFBlock(nn.Module):
    def __init__(self, channels, reduction=16):
        super().__init__()
        self.scorer = ComplexityScorer(channels)
        self.channel_att = ChannelAttention(channels, reduction)
        self.spatial_att = SpatialAttention()

    def forward(self, x):
        # Step 1: compute complexity scores from feature map statistics
        score_c, score_s = self.scorer(x)

        # Reshape scores for broadcasting: [B] -> [B, 1, 1, 1]
        score_c = score_c.view(-1, 1, 1, 1)
        score_s = score_s.view(-1, 1, 1, 1)

        # Step 2: apply each attention weighted by its score
        # score_c=0.9, score_s=0.1 -> mostly channel attention
        # score_c=0.8, score_s=0.9 -> both needed
        channel_out = score_c * self.channel_att(x)
        spatial_out = score_s * self.spatial_att(x)

        # Step 3: residual fusion
        return x + channel_out + spatial_out

```

پس از محاسبه امتیازها، متد $\text{view}(-1, 1, 1, 1)$ ابعاد بردار امتیازها را از حالت دوبعدی $[B]$ به ساختار چهاربعدی تبدیل می‌کند تا فرآیند Broadcasting پایتورچ میسر شود. سپس خروجی ماژول توجه کانالی در $score_c$ و خروجی ماژول توجه مکانی در $score_s$ ضرب می‌شود. در نهایت، تانسورهای حاصل به همراه تانسور ورودی اصلی به صورت باقی‌مانده (Residual) با یکدیگر جمع ریاضی می‌شوند:

$$Y = X + (score_c \otimes \text{ChannelAttention}(X)) + (score_s \otimes \text{SpatialAttention}(X))$$

این رویکرد تضمین می‌کند که اگر یک ماژول توجه سهم نویزی یا ناچیزی در یک اپوک داشته باشد، ضریب آن به سمت صفر میل کرده و گراف محاسباتی بدون تخریب ویژگی‌های پایه، جریان داده را عبور می‌دهد.

۳. ساختار کلی شبکه پیشنهادی (DAFResNet18)

ساختار نهایی مدل بر پایه استخوان‌بندی شبکه استاندارد ResNet18 طراحی شده است، با این تفاوت که بلاک‌های اختصاصی DAF به صورت راهبردی پس از هر یک از لایه‌های اصلی استخراج ویژگی قرار گرفته‌اند.

```

# -----
# ResNet18 + DAF
# -----
class DAFResNet18(nn.Module):
    def __init__(self, num_classes=10, reduction=16):
        super().__init__()

        resnet = models.resnet18(weights=None)

        # Adapt for CIFAR-10 (32x32)
        self.conv1 = nn.Conv2d(3, 64, kernel_size=3, stride=1, padding=1, bias=
        self.bn1 = resnet.bn1
        self.relu = resnet.relu
        self.maxpool = nn.Identity()

        # ResNet layers
        self.layer1 = resnet.layer1 # 64 channels
        self.layer2 = resnet.layer2 # 128 channels
        self.layer3 = resnet.layer3 # 256 channels
        self.layer4 = resnet.layer4 # 512 channels

        # DAF blocks – one per layer
        # Each one independently learns when to use Channel vs Spatial
        self.daf1 = DAFBlock(64, reduction)
        self.daf2 = DAFBlock(128, reduction)
        self.daf3 = DAFBlock(256, reduction)
        self.daf4 = DAFBlock(512, reduction)

        self.avgpool = nn.AdaptiveAvgPool2d(1)
        self.fc = nn.Linear(512, num_classes)

```

در این توپولوژی، هر یک از بلاک‌های daf1 تا daf4 به صورت کاملاً مستقل و بر اساس رزولوشن و ساختار آماری تانسور ویژگی در همان سطح، یاد می‌گیرند که توازن میان توجه کانالی و مکانی را تنظیم کنند. این رفتار پویا باعث می‌شود تا در لایه‌های ابتدایی (که فیچرها ساختار فضایی مشخص‌تری دارند) و لایه‌های عمیق (که فیچرها معنایی و کانالی‌تر هستند)، مدل به صورت بهینه و خودمختار سوئیچ کند. همچنین با اصلاح فیلتر اولیه به هسته 3×3 با گام ۱ و جایگزینی maxpool با لایه همانی (nn.Identity)، از ریزش اطلاعات مکانی در تانسور ورودی تصاویر ابعاد کوچک جلوگیری شده است.